ASP.NET Core MVC + 3-Layer Architecture

🎬 Assignment: Movie Management System (3-Layer Architecture)

📌 Objective

Design and develop a **Movie Management System** using:

- ASP.NET Core MVC
- Entity Framework Core (Code First)
- SQL Server
- 3-Layer Architecture

🧱 Architecture Overview

Students must implement the project using the following layers:

1. Data Layer (Class Library)

Responsible for:

- Entity Models
- DbContext
- Database interaction

2. Business Layer (Class Library)

Responsible for:

- Business logic
- Service interfaces & implementations

3. Presentation Layer (MVC Project)

Responsible for:

- Controllers
- Views
- User interaction

🎯 Functional Requirements

Students must implement **CRUD operations** for a Movie entity.

🎥 Movie Entity Fields

Design a Movie model with the following fields:

- MovieId (Primary Key)
- Title (Required)
- Genre
- ReleaseDate
- Duration (in minutes)
- Rating (e.g., IMDB rating)
- Language

📂 Tasks Breakdown

🔹 Task 1: Create Solution Structure

Create a solution with **3 projects**:

1. MovieApp.Data (Class Library)
2. MovieApp.Business (Class Library)
3. MovieApp.Web (ASP.NET Core MVC)

🔹 Task 2: Data Layer Implementation

In MovieApp.Data:

- Create Movie model class
- Create AppDbContext
- Configure EF Core with SQL Server

- Add DbSet for Movie
- Apply migrations and update database

👉 Concepts to apply:

- Code First Approach
- Data Annotations (Validation)

🔹 **Task 3: Business Layer Implementation**

In MovieApp.Business:

- Create Interface:
- IMovieService
- Define methods:
    - GetAllMovies()
    - GetMovieById(int id)
    - AddMovie(Movie movie)
    - UpdateMovie(Movie movie)
    - DeleteMovie(int id)
- Implement Service Class:
- MovieService

👉 Concepts to apply:

- Dependency Injection
- Separation of Concerns

🔹 **Task 4: Presentation Layer (MVC)**

In MovieApp.Web:

Create Controller:

- MovieController

Implement Actions:

- Index → List all movies
- Details → View movie details
- Create → Add new movie
- Edit → Update movie
- Delete → Remove movie

Create Views:

- Index.cshtml
- Create.cshtml
- Edit.cshtml
- Details.cshtml
- Delete.cshtml

👉 Concepts to apply:

- Razor Views
- Tag Helpers
- Model Binding
- Validation

🔹 **Task 5: Dependency Injection Setup**

- Register DbContext in Program.cs
- Register MovieService with Interface

🔹 **Task 6: Validation**

Apply validations such as:

- Title → Required
- Rating → Range (e.g., 1 to 10)

- Duration → Positive number

🔹 **Task 7: UI Enhancements (Optional)**

- Add Bootstrap styling
- Add search/filter by Genre
- Add sorting (by Rating or Release Date)

🧪 **Expected Output**

Students should be able to:

- Add a new movie
- View all movies
- Edit movie details
- Delete a movie
- See validation messages

🚀 **Bonus Challenges**

- Implement Repository Pattern
- Add Async methods (async/await)

❗ **Instructions**

- Do NOT mix business logic in controller
- Follow clean folder structure
- Use proper naming conventions
- Push code to GitHub